

VAJRA v0 — Technical Specification

Status: Phase 5 — Architecture Specification

Project: VAJRA

Purpose: Durable autonomous engineering runtime

Implementation status: Not yet implemented

Primary constraint: ₹0.00 operating cost

1. System Definition

VAJRA is a durable autonomous engineering runtime that accepts an engineering objective and attempts to produce a verified engineering artifact.

VAJRA is not:

- a chatbot
- an LLM wrapper
- an IDE
- a single autonomous agent
- a Telegram bot
- a model hosting platform
- a collection of shell scripts

VAJRA is a **durable execution system for engineering work**.

Its fundamental guarantee is:

A worker, model, process, network connection, or machine may disappear without destroying the Engineering Run.

The system must maintain canonical state independently from the model and workers.

2. Architectural Laws

These laws are normative.

Law 1 — Model Is Not Authority

A model may propose actions.

A model may never directly authorize or execute them.

```
Model
↓
Intent
↓
Policy
↓
Execution Broker
↓
Actual operation
```

Law 2 — VAJRA Owns Canonical State

Models, workers, Telegram, files, caches and external services are not authoritative state stores.

VAJRA's durable state is authoritative.

Law 3 — Meaningful Operations Are Durable

A meaningful engineering operation must have a durable representation.

Examples:

- planning
 - model invocation
 - patch application
 - command execution
 - verification
 - human approval
 - worker assignment
 - retry
 - recovery
-

Law 4 — Reasoning Cannot Directly Execute

The reasoning plane can produce structured intents.

It cannot directly call arbitrary shell commands, manipulate Git, access credentials, or modify the repository.

Law 5 — Worktree Isolation Is Not Security Isolation

Git worktrees provide logical repository isolation.

They do not provide:

- kernel isolation
- network isolation
- credential isolation
- process isolation
- filesystem security

Those responsibilities belong to the execution environment.

Law 6 — Verification Is External

A model saying:

"The task is complete."

is not evidence of completion.

Completion requires independently generated evidence.

Law 7 — Failure Is Data

Failures must be represented explicitly.

A failed attempt is not the same thing as a failed Engineering Run.

Law 8 — Autonomy Is Bounded

Every autonomous Run must have limits.

At minimum:

- time budget
- step budget
- retry budget
- execution budget
- model budget
- resource limits

- scope limits

There is no infinite retry loop.

Law 9 — Progress Requires Evidence

VAJRA must distinguish:

model believes progress happened

from:

verified artifact exists

Only the latter constitutes engineering progress.

Law 10 — Human Is Ultimate Authority

VAJRA may operate autonomously within explicitly granted policy.

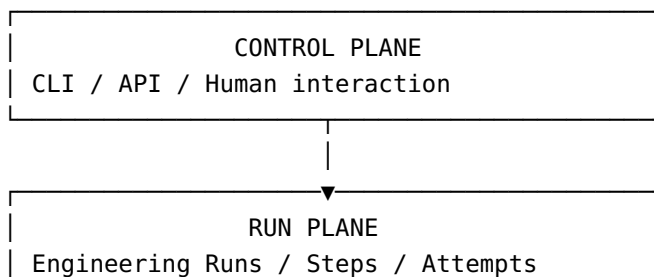
Certain actions must transition the Run into durable:

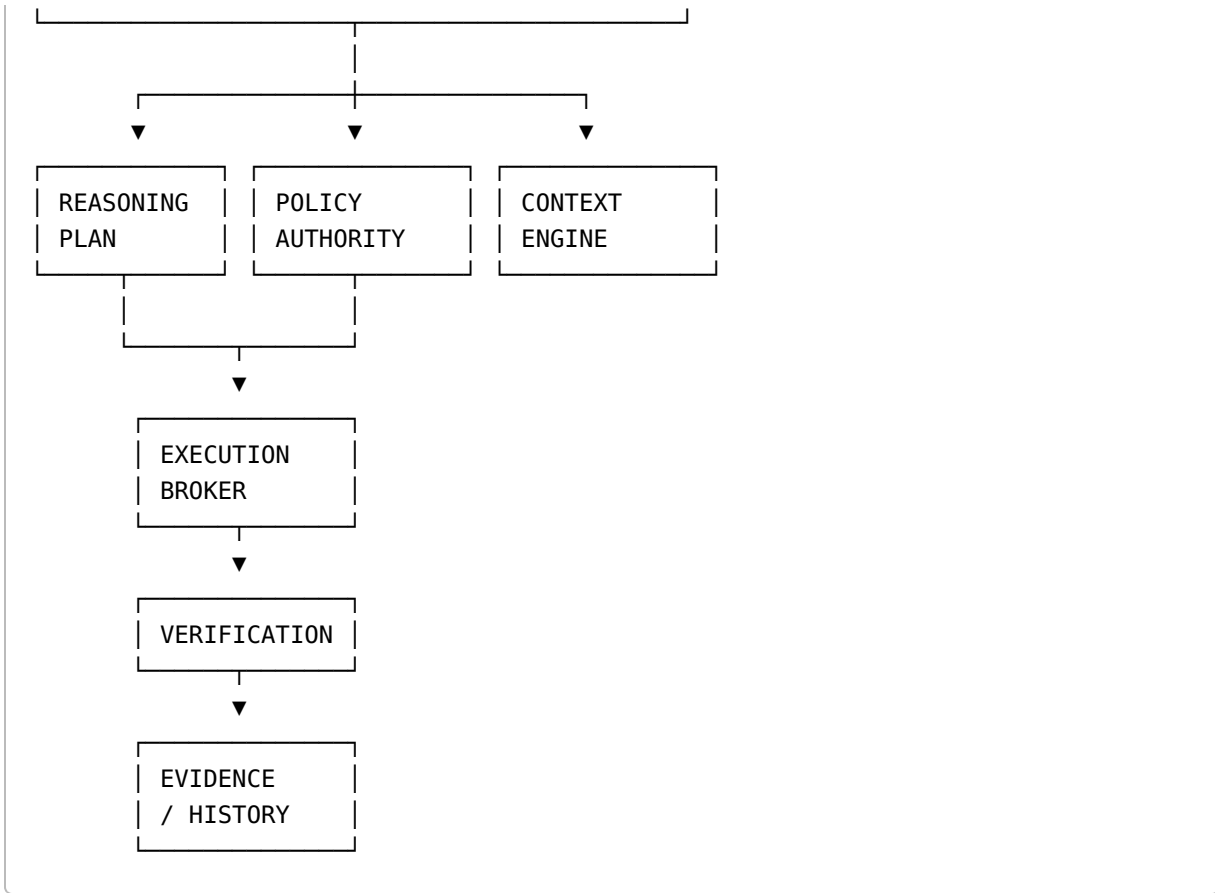
WAITING_HUMAN

state.

3. System Planes

VAJRA v0 contains seven logical planes.





4. Durable Engineering Run

The Engineering Run is the primary durable object.

```
EngineeringRun
├─ identity
├─ objective
├─ repository
├─ base_revision
├─ acceptance_criteria
├─ policy
├─ budget
├─ state
├─ steps
├─ attempts
├─ artifacts
├─ verification_results
```

```
|— events
|— final_disposition
```

Required Run fields

```
run_id
created_at
updated_at

objective

repository_id
repository_url
base_revision

acceptance_criteria

policy_id
policy_version

budget_id

state

workspace_id

current_step_id

created_by

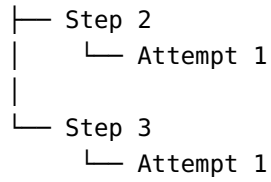
final_disposition
```

The exact persistence representation may be relational tables rather than a single serialized object.

5. Step Model

A Run consists of durable Steps.

```
Run
|— Step 1
|   |— Attempt 1
|   |— Attempt 2
|
```



A Step represents an intended unit of engineering progress.

Examples:

```
orient repository
inspect authentication module
modify configuration
run targeted tests
repair failing test
run complete verification
```

A Step survives worker failure.

6. Attempt Model

Attempts are disposable.

```
Step
├─ Attempt A
├─ Attempt B
└─ Attempt C
```

Each Attempt contains:

```
attempt_id
run_id
step_id

worker_id
model_id
model_version

started_at
finished_at

status
```

```
input_context_digest
output_digest
```

```
usage
errors
```

```
artifacts
verification_refs
```

If an Attempt dies:

```
Attempt → FAILED
Step → RECOVERING
Step → new Attempt
```

The Run remains intact.

7. Run State Machine

Primary lifecycle:

```
CREATED
↓
QUEUED
↓
ORIENTING
↓
PLANNING
↓
EXECUTING
↓
VERIFYING
↓
CANDIDATE
↓
PROMOTION
↓
COMPLETE
```

Exceptional states:

RECOVERING
PAUSED
WAITING_HUMAN
FAILED
ABORTED
EXPIRED

State rules

CREATED

Run exists but has not entered execution.

QUEUED

Run is waiting for scheduling.

ORIENTING

Repository/context discovery.

PLANNING

Model produces structured engineering plan/intents.

EXECUTING

Approved operations are performed.

VERIFYING

Independent verification is running.

CANDIDATE

Evidence indicates the requested outcome may have been achieved.

PROMOTION

Final policy-controlled promotion is occurring.

COMPLETE

Acceptance criteria have been satisfied and final disposition recorded.

RECOVERING

VAJRA is reconciling state after failure.

WAITING_HUMAN

Run requires human authorization.

FAILED

Run cannot continue under its current policy/budget.

ABORTED

Human/system explicitly stopped the Run.

EXPIRED

Run exceeded its allowed lifetime.

8. Event Model

VAJRA maintains an append-oriented event history.

Representative events:

```
RunCreated
RunQueued
StepCreated
AttemptStarted
WorkerAssigned
ModelInvocationStarted
ModelInvocationCompleted
IntentProposed
IntentDenied
IntentApproved
CommandStarted
CommandCompleted
PatchProposed
PatchApplied
VerificationStarted
VerificationCompleted
FailureDetected
FailureClassified
```

```
RetryScheduled
HumanApprovalRequested
HumanApprovalGranted
HumanApprovalDenied
CheckpointCreated
RecoveryStarted
RecoveryCompleted
ArtifactCreated
RunPaused
RunCompleted
RunFailed
```

Every event should contain correlation identifiers:

```
run_id
step_id
attempt_id
worker_id
timestamp
event_type
payload
```

The event history is an audit trail and recovery input.

9. Durable Runtime

Candidate

VAJRA v0 will target a durable workflow runtime abstraction with **Restate as the leading implementation candidate**.

VAJRA must not expose Restate-specific semantics throughout the application.

Instead:

```
VAJRA
  ↓
DurableRuntime interface
  ↓
Restate adapter
```

This preserves the ability to evaluate alternatives later.

Required runtime capabilities

The runtime must support:

- durable execution
- persisted workflow progress
- recovery after process failure
- retry
- idempotency
- durable timers
- concurrency control
- state persistence
- deterministic replay/recovery semantics
- worker/service failure recovery

A physical kill/restart Gate Test must pass before this dependency is frozen.

10. Model Gateway

VAJRA must be model-agnostic.

```
ModelGateway
├─ NeedleAdapter
├─ LocalLlamaCppAdapter
├─ OllamaAdapter
├─ KaggleWorkerAdapter
└─ FutureProviderAdapters
```

The gateway should expose a stable internal contract.

Conceptually:

```
ModelRequest
├─ model
├─ task
├─ context
├─ tools
├─ output_schema
├─ budget
└─ deadline
```

Result:

```
ModelResult
├─ status
├─ structured_output
├─ raw_output_reference
├─ usage
├─ model_identity
├─ errors
└─ evidence
```

VAJRA records model identity/version with every meaningful invocation.

11. Reasoning Contract

The model does not receive unrestricted execution authority.

The conceptual reasoning API is:

```
propose(ContextBundle) -> Intent[]
```

Possible Intent types:

```
InspectFile
SearchCode
InspectSymbol
ApplyPatch
CreateFile
DeleteFile
RunCommand
RunTest
RunBuild
RequestHuman
Finish
```

The model may propose.

It does not execute.

12. Policy Engine

Policy is deterministic.

```
Intent
↓
Policy Engine
↓
ALLOW
DENY
MODIFY
HUMAN_REQUIRED
```

Policy checks include:

- command
- path
- repository scope
- filesystem scope
- network capability
- credential capability
- destructive action
- resource limits
- Run budget
- worker capabilities
- sandbox requirements
- human approval requirements

Policy versions must be recorded with the Run.

13. Execution Broker

The Execution Broker is the only component permitted to convert approved operations into real execution.

```
Intent
↓
Policy
↓
Execution Broker
↓
Sandbox
```

↓
Tool / Shell / Git / Test

The model cannot bypass the broker.

No model-generated shell command may execute directly from the reasoning layer.

14. Worker Protocol

A Worker receives a bounded job.

```
WorkerJob
├─ run_id
├─ step_id
├─ attempt_id
├─ repository_revision
├─ workspace_contract
├─ context_bundle
├─ allowed_capabilities
├─ budget
├─ deadline
└─ expected_output_schema
```

Worker returns:

```
WorkerResult
├─ status
├─ structured_result
├─ artifacts
├─ logs
├─ usage
└─ errors
```

Workers do not own canonical Run state.

15. Worker Identity and Leasing

Workers must be treated as disposable.

Future multi-worker mode requires:

```
worker_id  
lease_id  
lease_expiry  
attempt_id
```

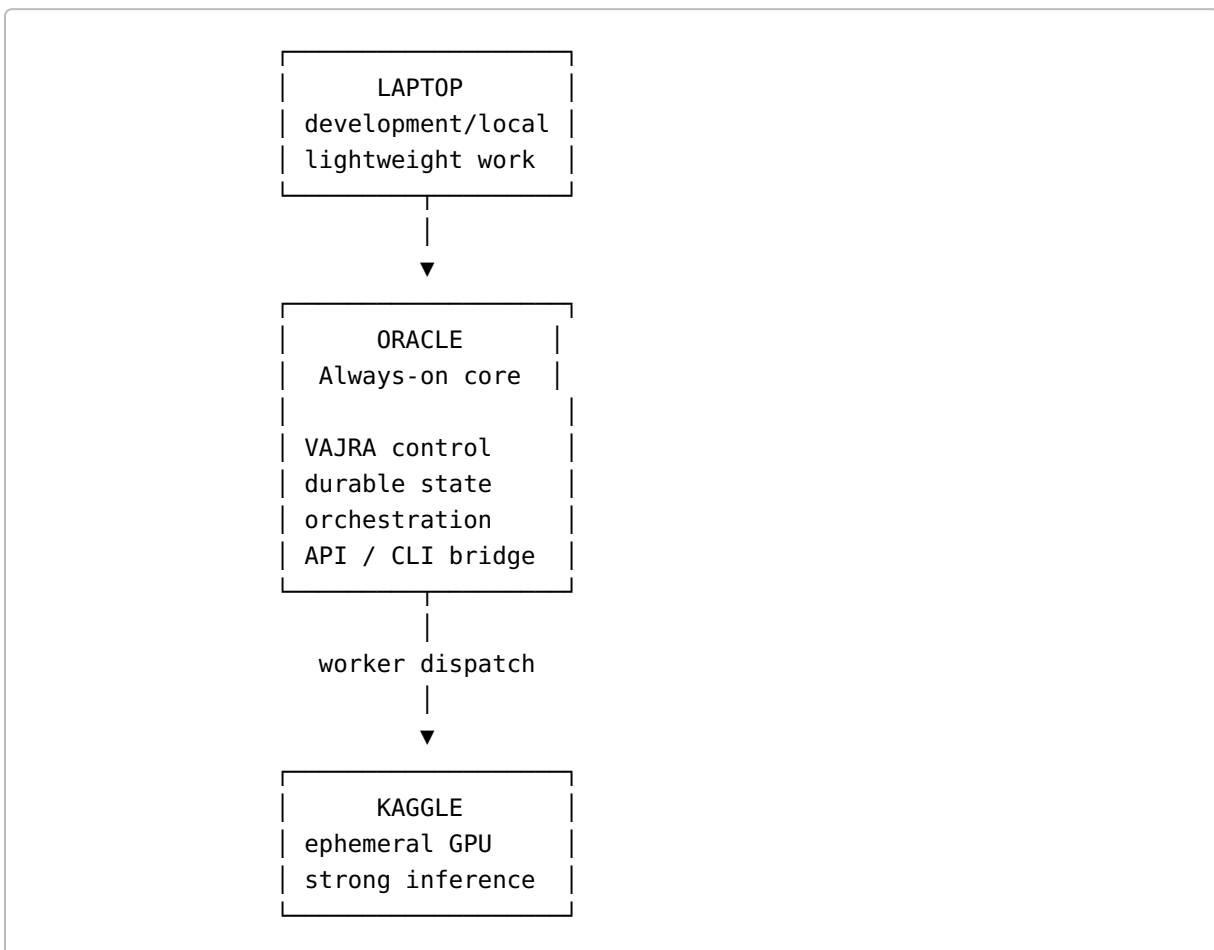
A stale worker must not be able to commit work after its lease expires.

Fencing/lease semantics are required before concurrent workers are enabled.

v0 may restrict execution to one active worker.

16. Laptop / Oracle / Kaggle Topology

VAJRA will use a three-location topology.



Oracle

Oracle is the durable control-plane host.

It should host:

- VAJRA services
- durable runtime
- durable Run state
- event history
- worker coordination
- API/control interface
- lightweight supporting services

Oracle is not assumed to be the heavy coding-model host.

Kaggle

Kaggle is an ephemeral compute worker.

It should perform:

- stronger model inference
- coding
- planning
- debugging
- other GPU-heavy tasks

Kaggle must not become the canonical state store.

If Kaggle disappears:

```
worker failure
↓
Attempt failure
↓
Run recovery
↓
retry / alternate worker / human
```

The Run remains durable.

Laptop

The laptop provides:

- development
 - local CLI
 - local testing
 - Needle
 - llama.cpp/Ollama
 - local experimentation
 - repository operations
-

17. Capability-Based Worker Selection

VAJRA must not assume a specific GPU.

Workers advertise capabilities.

```
WorkerCapabilities
├─ accelerator
├─ vram
├─ system_ram
├─ model
├─ model_version
├─ context_limit
├─ supported_tasks
├─ sandbox_type
└─ network_policy
```

Therefore VAJRA does not hardcode:

```
"P100 worker"
```

It asks:

```
"Which available worker satisfies this job?"
```

This is especially important because Kaggle's available accelerator inventory changes.

18. Workspace Model

Each Run receives a workspace.

Primary logical mechanism:

```
Git worktree
```

Conceptually:

```
repository
├─ main
├─ vajra-run-A
├─ vajra-run-B
└─ vajra-run-C
```

A workspace has:

```
workspace_id
run_id
base_revision
path
status
cleanup_state
```

Worktree state must be reconciled during recovery.

19. Sandbox Model

Sandboxing is a separate abstraction.

```
SandboxBackend
├─ RestrictedLocal
├─ Container
├─ gVisor
├─ Firecracker
└─ RemoteWorker
```

v0 should keep the interface abstract.

Target architecture for genuinely untrusted autonomous execution is a stronger isolation boundary than ordinary Git worktrees.

Potential candidates:

- gVisor
- Firecracker/microVM
- appropriately isolated remote worker

The actual backend is frozen only after the Gate Test.

20. Context Engine

VAJRA must not dump the entire repository into a model context.

Pipeline:

```
Repository
↓
Indexer
↓
tree-sitter / symbols
↓
references / dependency graph
↓
ranking
↓
ContextBundle
```

The Context Engine produces task-specific context.

21. ContextBundle

Conceptual structure:

```
ContextBundle
├─ objective
├─ acceptance_criteria
├─ repository_summary
├─ relevant_files
├─ relevant_symbols
```

```
|— dependencies
|— recent_changes
|— relevant_history
|— constraints
|— allowed_capabilities
└— budget
```

The model receives the smallest useful context rather than the entire repository.

22. Memory

VAJRA v0 intentionally does not begin with a vector database.

Initial persistent memory consists of:

- Run history
- events
- artifacts
- verification evidence
- failure history
- structured decisions
- relevant repository state

Principle:

History first. Memory later.

This reduces memory poisoning and unnecessary complexity.

23. Verification Engine

Verification is independent from the reasoning model.

Typical pipeline:

```
Static checks
  ↓
Targeted tests
  ↓
Build
  ↓
Full test suite
```

↓
Acceptance checks

Not every Run needs every layer.

Verification is determined by acceptance criteria and policy.

The verifier must produce structured evidence.

Example:

```
VerificationResult
├─ check_id
├─ command
├─ status
├─ exit_code
├─ output_reference
├─ environment
├─ timestamp
└─ evidence_digest
```

24. Completion Semantics

A Run cannot become **COMPLETE** because the model says it is done.

The transition requires:

```
objective
+
acceptance criteria
+
artifact
+
verification evidence
+
policy approval
```

Conceptually:

CANDIDATE
↓

Acceptance Evaluation

↓

PROMOTION

↓

COMPLETE

25. Failure Engine

Failure is represented explicitly.

Pipeline:

Failure

↓

Classifier

↓

Recovery Policy

↓

Action

Possible actions:

RETRY

NEW_STRATEGY

CHANGE_MODEL

CHANGE_WORKER

RESTORE_CHECKPOINT

REQUEST_HUMAN

ABORT

Failure categories may include:

MODEL_FAILURE

TOOL_FAILURE

COMMAND_FAILURE

TEST_FAILURE

POLICY_DENIAL

RESOURCE_EXHAUSTION

TIMEOUT

WORKER_LOSS

NETWORK_FAILURE

```
STATE_DIVERGENCE
SANDBOX_FAILURE
NO_PROGRESS
UNKNOWN
```

26. No-Progress Detection

VAJRA must detect cases where repeated attempts produce no meaningful advancement.

Examples:

```
same patch repeatedly proposed
same test repeatedly fails
same error repeated
repository unchanged across attempts
model oscillates between strategies
```

A Run must eventually stop or escalate rather than loop indefinitely.

27. Budgets

Every Run receives explicit budgets.

Minimum:

```
max_runtime
max_steps
max_attempts
max_model_calls
max_command_count
max_output_size
max_worker_runtime
```

Optional future budgets:

```
max_network_bytes
max_disk_usage
```

```
max_model_tokens
max_parallel_workers
```

Budget exhaustion must produce a durable state transition.

28. Checkpoint Model

A checkpoint records enough state to support recovery.

Conceptually:

```
Checkpoint
├─ checkpoint_id
├─ run_id
├─ step_id
├─ event_position
├─ git_revision
├─ workspace_identity
├─ policy_version
├─ state_digest
└─ evidence_refs
```

Git stash alone is insufficient as a complete checkpoint mechanism because untracked files and external state can escape it.

Checkpoint semantics therefore belong to VAJRA rather than being delegated entirely to Git.

29. Recovery

Recovery must reconcile actual state with durable state.

```
worker/process dies
↓
durable runtime detects interruption
↓
Run enters RECOVERING
↓
reconcile:
- current Run state
- event history
```

- workspace
- Git state
- checkpoint
- worker lease

↓

resume / retry / rollback / human

VAJRA must not blindly assume the external world matches its previous state.

30. Security Boundary

VAJRA treats the following as untrusted:

- model output
- repository contents
- repository instructions
- dependencies
- generated scripts
- worker environment
- external tools
- network responses
- persisted model-generated memory

Threat classes include:

- prompt injection
- tool injection
- command injection
- path traversal
- symlink attacks
- credential exposure
- dependency attacks
- network exfiltration
- sandbox escape
- verification manipulation
- memory poisoning
- stale-worker execution

31. Credentials

Secrets must not be placed directly into normal model context.

Credentials should be:

- scoped
- capability-bound
- injected only when required
- unavailable to untrusted operations unless explicitly authorized

A future secret broker may provide short-lived credentials.

v0 should minimize secret requirements.

32. MCP

MCP is treated as a tool interoperability protocol.

It is not:

- an authorization system
- a security boundary
- the canonical state system
- the execution authority

Correct architecture:

```
Model
↓
MCP/tool request
↓
VAJRA policy
↓
Execution Broker
↓
authorized capability
↓
tool
```

33. Human Control

Human interaction is an interface layer.

Potential interfaces:

```
CLI
API
Telegram
Web UI
```

Telegram is not the autonomous control plane.

A human approval must become durable state:

```
WAITING_HUMAN
```

and survive process/worker failure.

34. Observability

Every significant action should be traceable through:

```
run
step
attempt
worker
model
tool
workspace
artifact
verification
```

The system should answer:

What happened?

Why did it happen?

Which model proposed it?

Which policy allowed it?

Which worker executed it?

What evidence proves the result?

35. ASTRA Reuse Policy

ASTRA remains a reference implementation.

VAJRA will not import the ASTRA legacy core wholesale.

Adapt

Command policy	→ ADAPT
Patch safety	→ ADAPT
Checkpoint concepts	→ ADAPT
Git handling	→ INVESTIGATE

Investigate

Detection
Validation
Rollback
Transaction model
Security model

Reject

ASTRA legacy core

The reason is architectural coupling and its role as a compatibility/behavior anchor.

VAJRA should establish cleaner boundaries rather than reproducing the old central core.

36. v0 Scope

VAJRA v0 deliberately remains small.

Required:

- one repository
- one active Run
- one active worker
- durable Run

- durable Steps
- Attempts
- model adapter
- context bundle
- Git worktree
- policy engine
- execution broker
- isolated execution
- patch application
- command policy
- verification
- retry
- budgets
- human escalation
- event history
- CLI

Not required for v0:

- multiple concurrent Runs
- distributed worker pool
- Telegram
- web UI
- vector database
- autonomous memory learning
- complex multi-agent collaboration
- automatic model fine-tuning
- Kubernetes
- Temporal migration
- sophisticated observability platform

37. v1 Expansion

After v0 proves the architecture:

```
remote workers
Kaggle adapter
Oracle deployment
multiple concurrent Runs
worker leasing
stronger sandbox
richer context engine
failure learning
```

Telegram control
observability

38. Model Roles

Needle

Primary role:

local lightweight specialist

Suitable for:

- classification
- routing
- structured extraction
- simple tool selection
- lightweight argument generation

Not the primary coding model.

Needle confidence is informational.

It does not grant authorization.

Local llama.cpp / Ollama

Suitable for:

- local inference
 - summarization
 - context compression
 - inexpensive reasoning
 - mechanical tasks
 - local fallback
-

Kaggle

Suitable for:

- strong coding models
- complex planning
- debugging
- GPU-heavy inference

Kaggle workers are disposable.

39. Zero-Rupee Deployment Rule

VAJRA's production architecture must not depend on paid inference or paid infrastructure.

Target:

```
Laptop
+
Oracle Always Free
+
Kaggle free GPU
+
free/open-source software
```

The system must enforce resource boundaries so that free-tier assumptions do not silently become paid usage.

No automatic cloud upgrade should be part of VAJRA.

40. Gate Tests

Before full implementation, the following tests must pass.

Gate A — Durable Recovery

Prototype:

```
start Run
↓
```

```
execute Step 1
↓
persist checkpoint
↓
kill process/runtime
↓
restart
↓
recover
```

Acceptance:

- Run survives
- completed Step is not incorrectly rerun
- incomplete Step is recoverable
- state remains consistent
- event history remains coherent

Required: PASS.

Gate B — Needle Benchmark

Measure on the actual laptop:

- startup time
- RAM usage
- latency
- classification accuracy
- structured-output reliability
- tool-selection accuracy
- argument hallucination rate

Acceptance:

Needle must provide enough value to justify keeping it as a local specialist.

It does not need to replace a coding model.

Gate C — Kaggle Worker Lifecycle

Prototype:

```
Oracle
↓
dispatch job
↓
Kaggle worker
↓
perform inference
↓
return structured result
↓
worker disappears
```

Also test:

```
Oracle
↓
dispatch
↓
Kaggle dies
↓
detect failure
↓
recover Attempt
```

Acceptance:

- Oracle retains canonical state
- worker loss does not destroy Run
- result can be correlated to Attempt
- failed worker cannot later mutate the Run
- retry is possible

Gate D — Sandbox Isolation

Test candidate sandbox.

Attempt controlled:

- filesystem escape
- path traversal
- symlink escape
- unauthorized process access
- unauthorized network access

- credential visibility
- resource exhaustion

Acceptance:

Autonomous execution must not be able to escape its defined authority.

If gVisor fails the required deployment/security test, evaluate Firecracker.

Gate E — ASTRA Safety Extraction

Extract disposable test cases from ASTRA for:

- command policy
- patch safety
- checkpoint handling
- rollback
- detection
- validation

Acceptance:

Only proven behavior is carried into VAJRA.

No ASTRA architectural dependency may remain.

41. Gate Test Decision Rule

A gate produces one of:

PASS
PASS WITH LIMITATIONS
REWORK
REJECT

A candidate cannot become a frozen dependency merely because its documentation looks good.

Physical behavior wins over architectural assumptions.

42. Architecture Freeze Rule

After Gate Tests:

PASS

↓

freeze component

PASS WITH LIMITATIONS

↓

freeze with explicit constraint

REWORK

↓

return to Phase 5

REJECT

↓

select/test alternative

43. Implementation Boundary

Only after the Gate Tests pass does Phase 6 begin.

Phase 6 begins with:

VAJRA repository

↓

schemas

↓

durable runtime

↓

Run lifecycle

↓

worker protocol

↓

policy

↓

execution

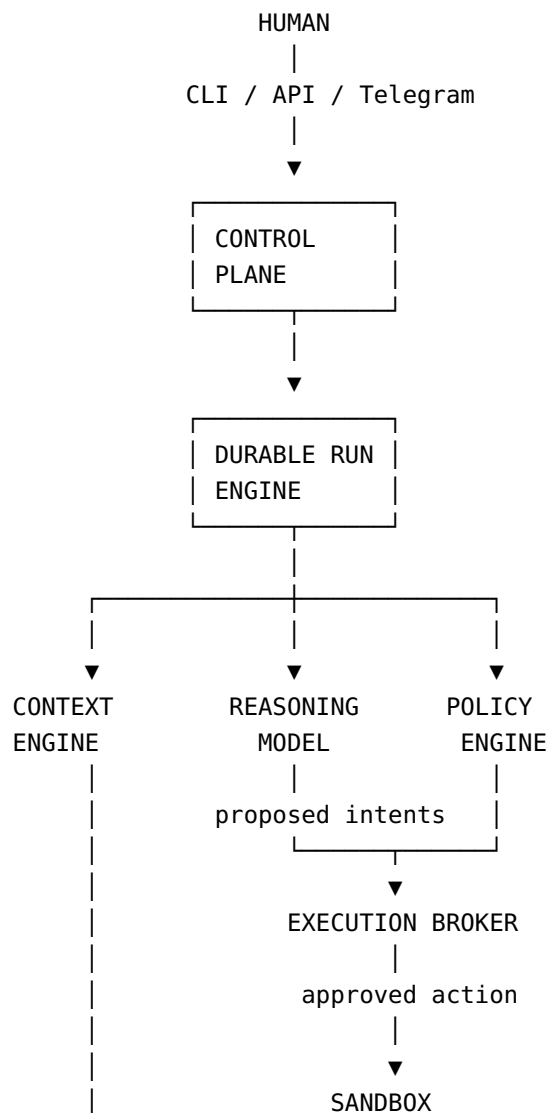
↓
verification
↓
recovery

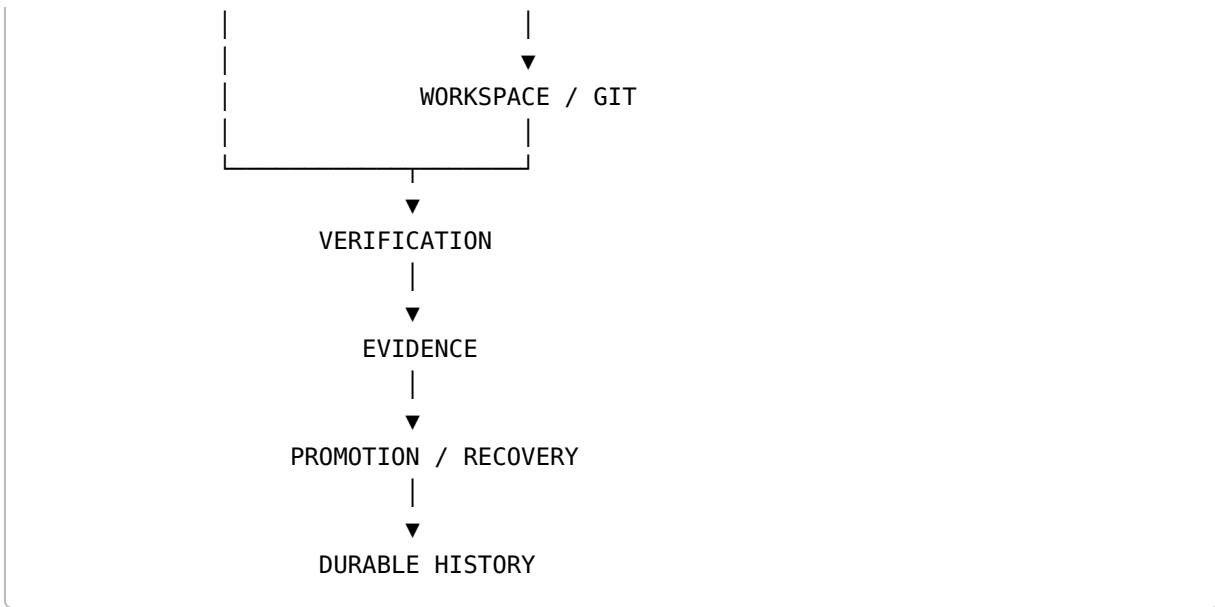
No feature should be implemented merely because it is interesting.

Every implementation must trace back to this specification.

44. Final v0 Architecture

The resulting conceptual architecture is:





45. The Most Important Boundary

The architecture ultimately reduces to this:



The model is therefore **inside the system, but never above the system.**

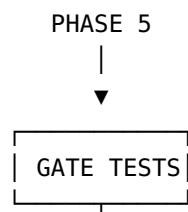
46. Phase 5 Exit Criteria

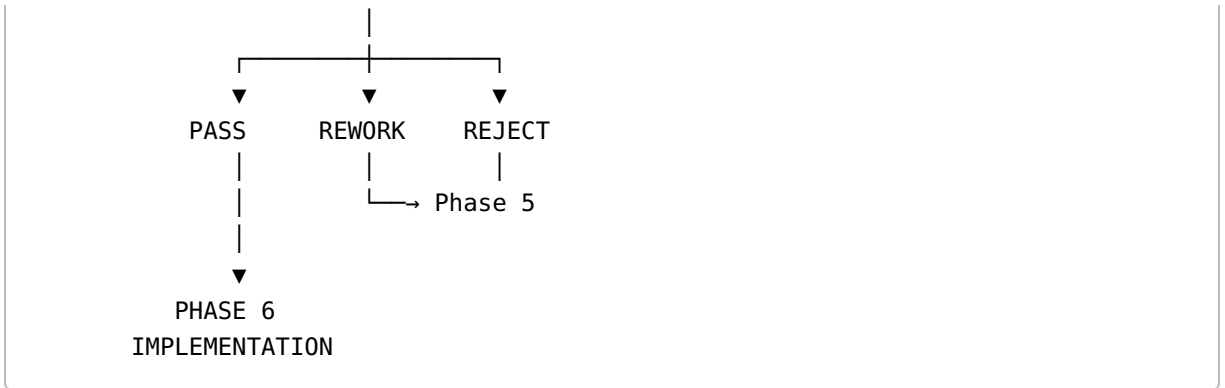
Phase 5 is complete when:

- [x] VAJRA definition frozen
- [x] architectural laws frozen
- [x] system planes defined
- [x] Engineering Run defined
- [x] Step/Attempt hierarchy defined
- [x] Run state machine defined
- [x] event model defined
- [x] durable runtime boundary defined
- [x] model gateway defined
- [x] worker protocol defined
- [x] policy boundary defined
- [x] execution boundary defined
- [x] workspace model defined
- [x] sandbox boundary defined
- [x] context model defined
- [x] verification semantics defined
- [x] failure/recovery semantics defined
- [x] budgets defined
- [x] security boundary defined
- [x] Oracle/Kaggle roles defined
- [x] ASTRA reuse policy defined
- [x] v0 scope defined
- [x] Gate Tests defined

Phase 5 output: Technical Specification

Next:





No production VAJRA implementation should begin before these gates are run.